

NCERT Solutions For Class 12 Computer Science

Chapter 5 – Sorting

1. Consider a list of 10 elements:

numList = [7, 11, 3, 10, 17, 23, 1, 4, 21, 5].

Display the partially sorted list after three complete passes of Bubble sort.

Answer: After each complete pass of Bubble sort, the largest unsorted element is placed at its correct position towards the end of the list.

Original List: [7, 11, 3, 10, 17, 23, 1, 4, 21, 5]

After Pass 1: [7, 3, 10, 11, 17, 1, 4, 21, 5, 23]

After Pass 2: [3, 7, 10, 11, 1, 4, 17, 5, 21, 23]

After Pass 3: [3, 7, 10, 1, 4, 11, 5, 17, 21, 23]

2. Thus, after three passes: [3, 7, 10, 1, 4, 11, 5, 17, 21, 23]

Identify the number of swaps required for sorting the following list using selection sort and bubble sort and identify which is the better sorting technique with respect to the number of comparisons.

List 1:

63	42	21	9
----	----	----	---

Answer:

Selection Sort:

Pass 1: 9 is placed at index 0 (swap 63 and 9) $\rightarrow [9, 42, 21, 63]$

Pass 2: 21 is placed at index 1 (swap 42 and 21) $\rightarrow [9, 21, 42, 63]$

Pass 3: 42 at index 2 (no swap needed)

Total Swaps: 2

Bubble Sort:
Pass 1: $[42, 21, 9, 63]$ (swap 63-42, 42-21, 21-9): 3 swaps

Pass 2: $[21, 9, 42, 63]$ (swap 42-21, 21-9): 2 swaps

Pass 3: $[9, 21, 42, 63]$ (swap 21-9): 1 swap

Total Swaps: 6

Selection sort performs fewer swaps (2) than bubble sort (6). In terms of number of comparisons, both perform similar comparisons (6 for four elements), but selection sort is better regarding swaps.

3. Consider the following lists:

List 1:

2	3	5	7	11
---	---	---	---	----

List 2:

11	7	5	3	2
----	---	---	---	---

If the lists are sorted using Insertion sort, which of the lists List1 or List2 will make the minimum number of comparisons? Justify using diagrammatic representation.

Answer:

Insertion Sort on List 1 (Already Sorted):

Pass 1: Compare 3 with 2 (1 comparison, no shift)

Pass 2: Compare 5 with 3 (1), with 2 (no shift)

Pass 3: Compare 7 with 5 (1), (no shift)

Pass 4: Compare 11 with 7 (1), (no shift)

Total comparisons: 4

List 1 requires the minimum comparisons as it is already sorted (best case for insertion sort). Insertion Sort on List 2 (Reversely Sorted):

Pass 1: Compare 7 with 11 (1, need to shift)

Pass 2: 5 with 7, 11 (2, need to shift both)

Pass 3: 3 with 5, 7, 11 (3, shift all three)

Pass 4: 2 with 3, 5, 7, 11 (4, shift all four)

Total comparisons: 10

List 2 requires maximum comparisons and shifts (worst

case).Conclusion: List 1 will make the minimum number of comparisons (best case for insertion sort), while List 2 (reverse order) will take the maximum.

4. Write a program using user defined functions that accepts a List of numbers as an argument and finds its median. (Hint: Use bubble sort to sort the accepted list. If there are odd number of terms, the median is the center term. If there are even number of terms, add the two middle terms and divide by 2 get median)

Answer:

```
def bubble_sort(lst):
    n = len(lst)
    for i in range(n):
        for j in range(0, n-i-1):
            if lst[j] > lst[j+1]:
                lst[j], lst[j+1] = lst[j+1], lst[j]

def find_median(lst):
    bubble_sort(lst)
    n = len(lst)
    if n % 2 == 1:
        return lst[n // 2]
    else:
        return (lst[(n // 2) - 1] + lst[n // 2]) / 2
```

```
# Example usage:
numbers = [7, 11, 3, 10, 17]
result = find_median(numbers)
print("Median is:", result)
# For odd length, returns middle value. For even, average
of two middle values.
```

5. All the branches of XYZ school conducted an aptitude test for all the students in the age group 14 – 16. There were a total of n students. The marks of n students are stored in a list. Write a program using a user defined function that accepts a list of marks as an argument and calculates the 'xth' percentile (where x is any number between 0 and 100). You are required to perform the following steps to be able to calculate the 'xth' percentile.

Note: Percentile is a measure of relative performance i.e. It is calculated based on a candidate's performance with respect to others. For example : If a candidate's score is in the 90th percentile, that means she/he scored better than 90% of people who took the test.

(I) Order all the values in the data set from smallest to largest using Selection Sort. In general any of the sorting methods can be used.

(II) Calculate index by multiplying x percent by the total number of values n.

For example: to find 90th percentile for 120 students: $0.90 * 120 = 108$

(III) Ensure that the index is a whole number using `math.round()`.

(VI) Display the value at that index — it is the xth percentile.

The corresponding value in the list is the xth percentile.

Answer:

```
import math

def selection_sort(lst):
    n = len(lst)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if lst[j] < lst[min_idx]:
                min_idx = j
        lst[i], lst[min_idx] = lst[min_idx], lst[i]

def percentile(lst, x):
    n = len(lst)
    selection_sort(lst)
    index = int(round((x / 100) * n))
    if index == 0:
```

```

        index = 1
    if index > n:
        index = n
    return lst[index-1]

# Example usage:
marks = [45, 67, 89, 34, 56, 78, 90]
x = 90
result = percentile(marks, x)
print(f"{x}th percentile is:", result)

```

6. During admission in a course, the names of the students are inserted in ascending order. Thus, performing the sorting operation at the time of inserting elements in a list. Identify the type of sorting technique being used and write a program using a user defined function that is invoked every time a name is input and stores the name in ascending order of names in the list.

Answer: The sorting technique being used is **Insertion Sort**, as each new element (name) is inserted at its correct position to maintain the order.

```

def insert_name(sorted_list, name):
    sorted_list.append(name)
    i = len(sorted_list) - 2
    while i >= 0 and sorted_list[i] > name:
        sorted_list[i + 1] = sorted_list[i]

```

```
        i -= 1
    sorted_list[i + 1] = name

# Example usage:
names = []
insert_name(names, "Amit")
insert_name(names, "Neha")
insert_name(names, "Bharat")
print(names)    # Output: ['Amit', 'Bharat', 'Neha']
```